Denis Melnikov

+7 931 511 58 76
melnikov.denis.aleksandrovich@gmail.com
@daikofnetu
Dinichthys
Moscow, Russia

Education

Bachelor's Degree | Second Year, GPA: 8.65 — Sep. 2025 – Present
School of Radio Engineering and Computer Technologies
Moscow Institute of Physics and Technology — Dolgoprudny, Russia

Additional Courses

Course by Ilya Rudolfovich Dedinsky on C and Assembly Languages — Sep. 2024 – May 2025
Course by Ilya Rudolfovich Dedinsky on C++ — Sep. 2025 – May 2026
Introduction to Computer Systems Architecture by SBER — Feb. 2026 – May 2026
Introduction to Tensor Compilers by SBER — Feb. 2026 – May 2026

Work Experience

Go Compiler Development Team Experience — July – September 2025

Intern (Assistant Engineer) at Huawei, full-time.
I enhanced a specific optimization pass in the compiler to handle more cases. As a result of my internship, my commit to the official compiler repository was accepted and the changes were merged.

Teaching Experience

Mentorship Experience on Ilya Rudolfovich Dedinsky's Course — Sep. 2025 – May 2026

I worked as a mentor on Ilya Rudolfovich Dedinsky's C and Assembly Languages course for first-year students throughout the full academic year.

Projects

Compiler | C, CMake, NASM — Compiler on GitHub

Project Goal: Implement a compiler for a custom programming language targeting a custom architecture, as well as NASM. The compiler front-end tokenizes the source code and builds an Abstract Syntax Tree (AST) using recursive descent parsing, which is then converted to an Intermediate Representation (IR).

The IR is then optimized by the middle-end, which performs the following optimizations:

- Dead Code Elimination: Removes unreachable code.
- Common Subexpression Elimination: Computes identical subexpressions only once.
- Unused Variable Elimination: Removes unused variables from the program.

Finally, the back-end translates the IR into assembly instructions for a functional simulator, which subsequently converts them to machine code and executes them.

LLVMPass | C++, CMake — LLVMPass on GitHub

Project Goal: Implement an LLVM pass that constructs Control Flow Graphs (CFG) and Data Flow Graphs (DFG), instruments LLVM IR with logging instructions, collects the resulting profile, and visualizes hot/cold functions and basic blocks on the graph using colors and call counts.

Physics | C++, CMake — Physics on GitHub

Project Goal: Create a physics constructor for various objects with support for light reflection and refraction, as well as plugin extensibility. The final version included two plugins: a graphics library plugin (allowing switching between SFML and SDL without code changes) and a drawing overlay plugin (for rendering various shapes on top of the screen).

My plugin implementations can be found in the repositories: DR4Backend (graphics library plugin) and GeomPrimitiveBackend (drawing overlay plugin).

SPU | C, Make — SPU on GitHub

Project Goal: Implement a functional processor simulator with three components:

- Assembler: Translates assembly instructions into binary representation.
- Disassembler: Restores assembly instructions from binary representation.
- Processor: Executes instructions described in binary format.

Printf | Assembly (NASM), Make — Printf on GitHub

Project Goal: Implement a simplified version of the C printf function in assembly language supporting the following format string specifiers: %c, %s, %d, %b, %o, %x, and %f. Specifier parsing is performed using a JUMP table.
Additional Implemented Features:

- File Output: Supports writing formatted output to files.
- Colored Output:
  * For stdout: ANSI escape sequences used for color changes.
  * For file output: HTML-style tags used for text coloring (e.g., <font color="red">)
- Floating-Point Support: Implements the %f specifier for floating-point numbers.

Mandelbrot Set | C, CMake — Mandelbrot Set on GitHub

Project Goal: Optimize a program that renders the Mandelbrot set using various techniques.
Three Optimization Methods Applied:

- Array Optimization: Strategic array usage enabled the compiler to replace operations with SIMD instructions.
- SIMD Intrinsics: Using intrinsics allowed parallel processing of multiple data elements per instruction.
- Loop Unrolling: Unrolling loops reduced data dependencies between instructions, keeping the processor pipeline fully utilized.

Interests

Compilers
Optimizations
Neural Networks
Operating Systems

Hard Skills

Programming Languages: C, C++, Assembly (NASM, TASM, x86-64), Python, Go
Tools: CMake, Make, IDA, Ghidra, Radare2, edb, GDB

Languages

Russian: Native
English: B1 (Intermediate)

Soft Skills

Strong Communication Skills
Self-Confident
Responsible